Arquitectura de nube y AWS – Avance #4

Para este avance se dará a ver cuales son los beneficios, costos, dinámicas del uso de la información, funciones y flujos de datos en la arquitectura de la nube que permite mejorar la recolección de información en tiempo real.

- Beneficios

Una base de datos en la nube ayuda a que el sistema sea escalable, disponible y provea buena calidad de datos ya sea tanto para la directiva como lo seria una prueba de entrega o seguimiento de un paquete para un usuario. Además de tener acceso a la elasticidad en caso de aumento de deliveries en temporadas de navidad o viernes negro, también ayuda a la separación de cómputo y almacenamiento. Como lo es Amazon S3, que se puede usar como un *Data Lake* inmenso y barato, y conectar motores analíticos como Amazon Redshift (o desplegar **Snowflake** de forma nativa sobre la infraestructura de AWS). Diferentes equipos pueden hacer consultas complejas al mismo tiempo sin que la base de datos se vuelva lenta.

- Código Python

1. Lo que hace el script de Python es automatizar la creación y configuración de casi toda la arquitectura AWS. Se usa la librería boto3 para conectarse a los servidores de AWS.
2. Después de eso crea la base de datos relacional con “def crear_rds_postgresql():”. Aquí está aprovisionando una base de datos PostgreSQL usando el servicio **Amazon RDS**.
3. Lo que sigue es crear el data lake con la función “def crear_s3_bucket():” lo cual no solo crea el “bucket” sino que también la estructura de carpetas usando bucles “for”.
4. Despues de esto se crean las tablas rápidas para streaming usando DynamoDB que puede leer y escribir miles de datos por segundo con latencias muy bajas. ☐ Por eso las tablas se llaman deliveries_status (para actualizar la app al instante) o vehicle_tracking (para el GPS de los camiones).
5. Finalmente se configura la seguridad y orquestación. La función “def crear_rol_iam_lambda():”, dará permiso a futuras funciones de código (AWS Lambda) para que puedan leer datos de S3, escribir en DynamoDB o enviar alertas. Sin este rol, el proceso de extracción de tu ETL no tendría permiso para tocar los datos crudos.
6. Ya finalizando se genera dinámicamente un archivo Bash que contiene los comandos exactos para copiar los datos que la empresa tiene en el servidor local y pegarlos en la nube. Finalmente, la función main() actúa como el director de orquesta, ejecutando las funciones en orden y guarda un resumen de lo hecho.

- Funciones Lambda:

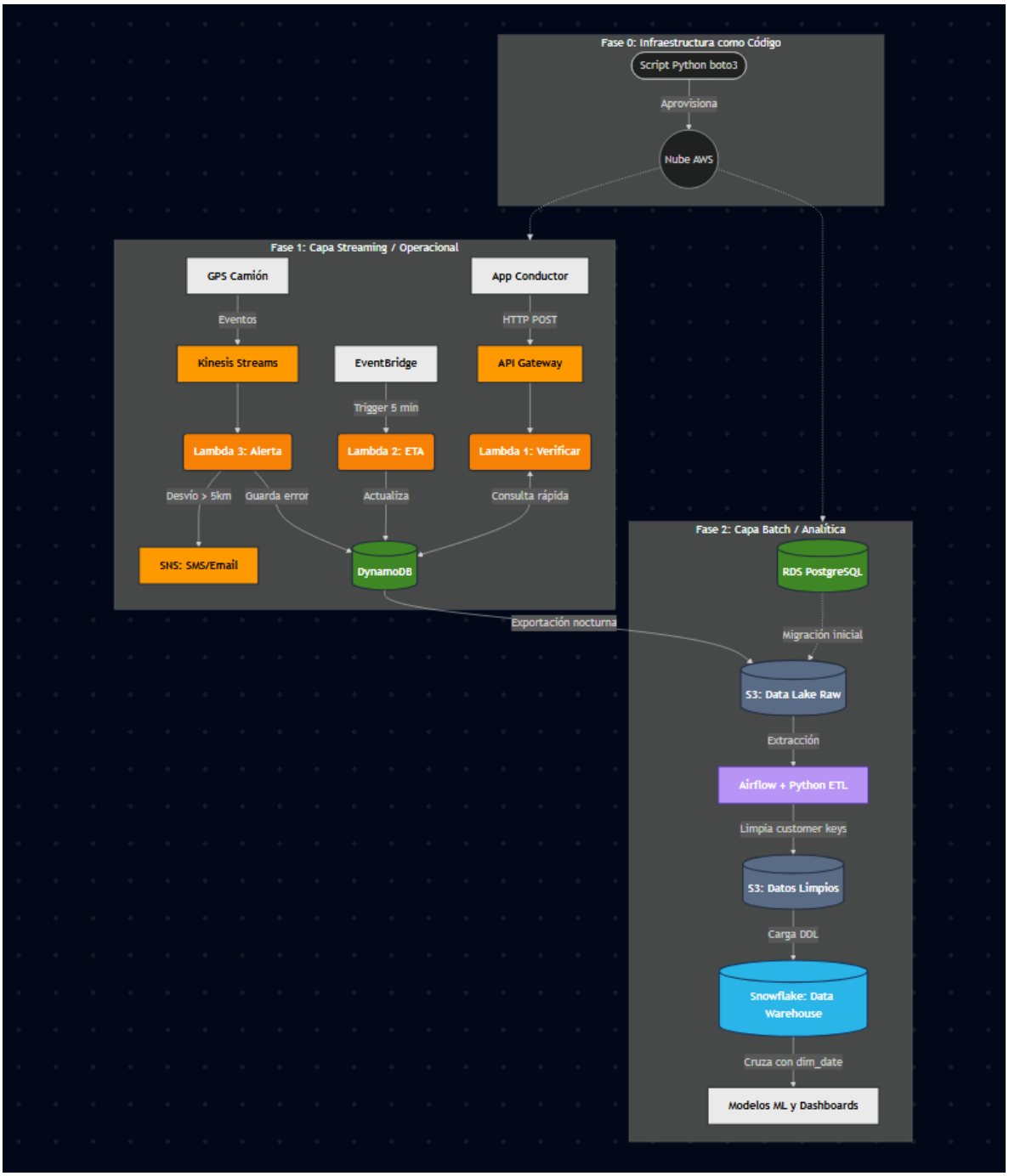
Lambda 1: lambda_verificar_entrega (El Micro-servicio Operacional) Está conectada al **API Gateway**. Cuando el cliente abre su aplicación para ver si su paquete ya llegó, la app hace una petición HTTP GET. Busca el ID del delivery y confirma, o no, la entrega .

Lambda 2: lambda_calcular_eta (El Micro-Lote o Micro-batch) Realiza matemáticas de coordenadas (fórmula de Haversine simplificada) para recalcular a qué hora llegará el camión. En lugar de esperar al día siguiente para analizar rutas, toma una foto del estado actual, hace el cálculo y actualiza la base de datos en tiempo real para que los clientes vean su ETA actualizado.

Lambda 3: lambda_alerta_desvio (El Verdadero Streaming) La función compara la ubicación actual con la ruta planeada (routes_waypoints). Si el camión se desvía más de 5 km, usa el servicio **Amazon SNS** para enviar inmediatamente un SMS, correo o notificación *push* al supervisor de flota y registra la alerta.

- Diagrama:

Para hacerlo mas digerible, adjunto diagrama



En resumen los servicios de AWS sirven de gran manera ya que ayudan a que mientras se dan los procesos de entrega la información fluya de manera rápida, eficaz y efectiva para las operaciones de los conductores, la verificación de entrega de los pedidos y la ingesta y almacenamiento de la información para que sea de posterior análisis como lo seria en la base de datos OLAP en snowflake